

**PONTIFÍCIA UNIVERSIDADE CATÓLICA DO PARANÁ
ESCOLA POLITÉCNICA
BACHARELADO EM ENGENHARIA DE COMPUTAÇÃO**

**ANDRÉ FABRICIO WOZNIACK
RICARDO HESPANHOL PAMPLONA**

**DETECÇÃO DE VULNERABILIDADES EM CÓDIGO-FONTE COM
MACHINE LEARNING**

PLANO DE PROJETO – 1º BIMESTRE

Orientador: Prof. Julio Cesar Nievola

Visto do Orientador

**CURITIBA
2026**

RESUMO

Palavras-chave: Palavra 1. Palavra 2. Palavra 3.

SUMÁRIO

1	INTRODUÇÃO	3
1.1	MOTIVAÇÃO E RELEVÂNCIA	3
1.2	OBJETIVOS	3
1.2.1	Objetivo Geral	3
1.2.2	Objetivos Específicos	4
1.3	ESTRUTURA DO DOCUMENTO	4
2	DETALHAMENTO DO PROBLEMA	5
3	ESTADO DA ARTE	6
3.1	FERRAMENTAS DE ANÁLISE ESTÁTICA	6
3.2	MODELOS TRADICIONAIS DE <i>MACHINE LEARNING</i>	7
3.3	<i>DEEP LEARNING</i> PARA ANÁLISE DE CÓDIGO	8
3.4	MODELOS PRÉ-TREINADOS PARA CÓDIGO	8
3.5	<i>LARGE LANGUAGE MODELS</i> E SOLUÇÕES HÍBRIDAS	9
3.6	ANÁLISE CRÍTICA DAS SOLUÇÕES EXISTENTES	10
3.7	SÍNTESE DO CAPÍTULO	11
4	TRABALHO A SER DESENVOLVIDO	12
4.1	DESCRIÇÃO FUNCIONAL E DIAGRAMA EM BLOCOS	12
4.2	DETALHAMENTO DOS MÓDULOS FUNCIONAIS	12
5	TECNOLOGIAS QUE SERÃO UTILIZADAS	13
5.1	TECNOLOGIAS ESCOLHIDAS E JUSTIFICATIVAS	13
5.2	TECNOLOGIAS ALTERNATIVAS AVALIADAS	13
6	PROCEDIMENTOS DE TESTE E VALIDAÇÃO DO PROJETO	14
6.1	PROCEDIMENTOS DE TESTE	14
6.2	CRITÉRIOS DE ACEITE E DESEMPENHO	14
6.3	ANÁLISE DOS RISCOS	14
7	CRONOGRAMA DO PROJETO	15
8	CONCLUSÃO	16
	REFERÊNCIAS	17

1 INTRODUÇÃO

1.1 MOTIVAÇÃO E RELEVÂNCIA

Com o aumento do uso de Inteligência Artificial para gerar código, trechos produzidos automaticamente vêm sendo incorporados a projetos sem análise adequada de segurança, o que pode introduzir vulnerabilidades como injeção de comandos, vazamento de dados e falhas de gerenciamento de memória. Estudos recentes indicam que código assistido por IA apresenta, em média, 4,4 vezes mais vulnerabilidades que código escrito apenas por humanos, com certos tipos de falha aparecendo com maior frequência e templates inseguros se repetindo em diferentes repositórios, sugerindo padrões induzidos pelos próprios modelos. Com isso surge uma necessidade de se tomar uma medida proativa para identificar vulnerabilidades em código-fonte antes de sua integração a um sistema, de forma a reduzir o risco de exploração de falhas de segurança e proteger dados sensíveis de usuários e empresas.

Nota-se também que embora existam ferramentas de análise estática ajudam a identificar parte desses problemas elas frequentemente geram falsos positivos e possuem dificuldade para compreender o contexto do código, como a intenção do desenvolvedor e as chamadas entre funções. Essa limitação é o que torna o problema relevante do ponto de vista técnico: não basta detectar vulnerabilidades já catalogadas por regras fixas, é preciso compreender o contexto do código para reduzir os falsos positivos e priorizar corretamente os alertas mais críticos.

1.2 OBJETIVOS

1.2.1 Objetivo Geral

Desenvolver um software capaz de analisar novos trechos de código-fonte antes de sua integração a um sistema, identificando padrões e erros comuns que possam resultar em vulnerabilidades de segurança conhecidas, utilizando técnicas de Machine Learning e Large Language Models para tornar essa análise mais contextual do que a oferecida pelas ferramentas tradicionais de análise estática.

1.2.2 Objetivos Específicos

Para alcançar o objetivo geral, foram definidos os seguintes objetivos específicos:

- Realizar uma revisão bibliográfica sobre vulnerabilidades de segurança em código-fonte, ferramentas de análise estática e técnicas de Machine Learning e Large Language Models aplicadas à detecção de vulnerabilidades;
- Definir a arquitetura do sistema, incluindo a decomposição em módulos e o diagrama em blocos do software;
- Selecionar as tecnologias que serão utilizadas para o desenvolvimento do software, justificando suas escolhas;
- Implementar o software, integrando os módulos e garantindo sua funcionalidade;
- Realizar testes e validação do software, avaliando sua eficácia na detecção de vulnerabilidades em código-fonte.

1.3 ESTRUTURA DO DOCUMENTO

Este documento está estruturado da seguinte forma: a Seção 2 detalha o problema a ser resolvido; a Seção 3 apresenta o estado da arte das soluções existentes; a Seção 4 descreve o trabalho a ser desenvolvido, incluindo a decomposição em módulos e o diagrama em blocos do sistema; a Seção 5 apresenta as tecnologias que serão utilizadas; a Seção 6 descreve os procedimentos de teste e validação; a Seção 6.3 traz a análise dos riscos do projeto; a Seção 7 apresenta o cronograma; e a Seção 8 conclui o documento.

2 DETALHAMENTO DO PROBLEMA

Primeiramente para entender os problemas de segurança em código, precisamos definir a produção de código antes das ferramentas de Inteligência Artificial. O código é produzido por desenvolvedores humanos, que escrevem manualmente cada linha de código, utilizando suas habilidades e conhecimentos para criar software funcional. Esse processo envolve a compreensão dos requisitos do sistema, a implementação de algoritmos e estruturas de dados, e a realização de testes para garantir que o código funcione corretamente. No entanto, mesmo com desenvolvedores experientes, erros podem ocorrer, resultando em vulnerabilidades de segurança.

No entanto, estamos vivendo uma verdadeira revolução na produção de código com o advento de ferramentas de Inteligência Artificial, como Claude Code, Codex, Antigravity, Cursor e o GitHub Copilot, que utilizam modelos de linguagem avançados para auxiliar os desenvolvedores na escrita de código. Essas ferramentas podem gerar código automaticamente, sejam por trechos ou até mesmo abordagens como o uso de agentes para criar tudo através com base em descrições em linguagem natural. Embora essas ferramentas possam aumentar a produtividade e reduzir o tempo de desenvolvimento, elas também introduzem desafios de segurança.

Com esse advento, surgem preocupações sobre a qualidade e segurança do código gerado automaticamente. Estudos recentes indicam que código assistido por IA apresenta, em média, 4,4 vezes mais vulnerabilidades que código escrito apenas por humanos, com certos tipos de falha aparecendo com maior frequência e templates inseguros se repetindo em diferentes repositórios, sugerindo padrões induzidos pelos próprios modelos (State of Cyber; Wu *et al.*, 2026, 2025). Embora ferramentas de análise estática ajudem a identificar esses problemas, elas frequentemente geram falsos positivos e possuem dificuldade para compreender o contexto do código, como a intenção do desenvolvedor e chamadas entre funções (Yang *et al.*; TechScience, 2025, 2025). Por isso, há demanda por abordagens mais contextuais, baseadas em Machine Learning e Large Language Models, que vêm sendo aplicadas à detecção de vulnerabilidades e à priorização de alertas em código gerado automaticamente (Wang *et al.*; MDPI, 2025, 2026).

3 ESTADO DA ARTE

A detecção de vulnerabilidades em código-fonte é um problema relevante para a segurança de software, pois falhas introduzidas durante o desenvolvimento podem resultar em acessos indevidos, vazamento de informações, execução remota de código, negação de serviço e comprometimento de sistemas. Tradicionalmente, esse problema é tratado por revisão manual de código, testes de segurança e ferramentas de análise estática. Entretanto, tais abordagens podem demandar tempo, depender da experiência dos analistas e produzir uma quantidade elevada de alertas, incluindo falsos positivos.

Com o crescimento da complexidade dos sistemas e do volume de código produzido, diferentes abordagens baseadas em *Machine Learning* (ML), *Deep Learning* (DL) e modelos de linguagem passaram a ser propostas para automatizar e ampliar a identificação de vulnerabilidades. Essas soluções procuram aprender padrões associados a falhas de segurança a partir de exemplos de código vulnerável e não vulnerável (Chakraborty *et al.*; Li *et al.*, 2021, 2022).

3.1 FERRAMENTAS DE ANÁLISE ESTÁTICA

Antes da ampla aplicação de técnicas de *Machine Learning*, a principal estratégia para a identificação automática de vulnerabilidades era a análise estática. Essa abordagem inspeciona o código-fonte sem a sua execução, buscando padrões, regras ou fluxos de dados associados a falhas de segurança.

Ferramentas como SonarQube, Semgrep, CodeQL, Checkmarx e Fortify são utilizadas para identificar padrões relacionados a vulnerabilidades conhecidas, tais como o uso de funções inseguras, validação inadequada de entradas, injeção de comandos, injeção SQL, *buffer overflow* e problemas de gerenciamento de memória. Algumas dessas ferramentas também permitem a criação de regras personalizadas, adequadas às convenções e aos requisitos de segurança de uma organização.

A principal vantagem das ferramentas de análise estática está na explicabilidade dos resultados. Em geral, elas indicam o trecho de código associado ao alerta, descrevem a regra violada e podem fornecer orientações para a correção. Dessa forma,

são adequadas para revisão de código e para integração em pipelines de integração contínua e entrega contínua.

Entretanto, muitas dessas soluções dependem de regras manualmente definidas. Consequentemente, sua capacidade de detectar vulnerabilidades novas ou padrões que não foram previamente modelados pode ser limitada. Além disso, ferramentas estáticas podem gerar falsos positivos, isto é, alertas sobre trechos que aparentam ser vulneráveis, mas que não representam um risco real no contexto da aplicação. Essas limitações motivaram o uso de técnicas de *Machine Learning*, capazes de aprender padrões a partir de exemplos de código.

3.2 MODELOS TRADICIONAIS DE *MACHINE LEARNING*

As primeiras pesquisas que aplicaram *Machine Learning* à detecção de vulnerabilidades utilizaram características extraídas manualmente do código-fonte. Essas características podem incluir quantidade de linhas de código, complexidade ciclomática, frequência de funções consideradas inseguras, operadores utilizados, chamadas de APIs, tokens e elementos da estrutura sintática do programa.

A partir dessas informações, algoritmos como *Random Forest*, *Support Vector Machine* (SVM), árvores de decisão, regressão logística e *Naive Bayes* podem ser treinados para classificar funções, arquivos ou módulos como vulneráveis ou não vulneráveis. Essas abordagens possuem, em geral, menor custo computacional quando comparadas a redes neurais profundas e podem ser mais simples de implementar e interpretar.

Entretanto, a eficácia desses modelos está diretamente relacionada à qualidade dos atributos selecionados. Como vulnerabilidades podem depender do fluxo de dados, das relações entre variáveis, das chamadas de funções e do contexto de execução, características superficiais nem sempre representam adequadamente o comportamento do programa. Portanto, modelos tradicionais podem apresentar dificuldade para detectar falhas complexas, especialmente quando o padrão vulnerável não está restrito a uma única linha ou função (Chakraborty *et al.*, 2021).

3.3 DEEP LEARNING PARA ANÁLISE DE CÓDIGO

Com o avanço das técnicas de *Deep Learning*, surgiram modelos capazes de aprender representações diretamente do código-fonte, reduzindo a necessidade de definir atributos manualmente. Entre as arquiteturas aplicadas ao problema estão redes neurais convolucionais, redes neurais recorrentes, LSTM, GRU, *transformers* e *Graph Neural Networks* (GNNs).

Uma solução conhecida é o Devign, proposto por Zhou et al. (Zhou et al., 2019). O método representa o código por meio de grafos e aplica redes neurais para identificar funções vulneráveis. Sua proposta considera não apenas tokens presentes no código, mas também relações estruturais e semânticas entre instruções. Essa estratégia é relevante porque uma vulnerabilidade pode decorrer da interação entre diferentes partes de uma função, e não apenas de uma instrução isolada.

Outra abordagem relevante consiste no uso de grafos de código, como *Abstract Syntax Trees* (ASTs), *Control Flow Graphs* (CFGs), *Data Flow Graphs* (DFGs) e *Code Property Graphs* (CPGs). Essas estruturas permitem representar, respectivamente, a sintaxe do programa, os possíveis caminhos de execução e o fluxo de informações entre variáveis. Modelos baseados em grafos podem explorar essas relações e são especialmente adequados para vulnerabilidades relacionadas à manipulação de memória, validação de entradas e uso incorreto de variáveis (Chakraborty et al., 2021).

Apesar dos resultados positivos, essas abordagens apresentam desafios. A construção dos grafos exige ferramentas adicionais para análise do código, e o custo computacional pode aumentar de acordo com o tamanho e a complexidade do projeto. Além disso, diversos estudos avaliam os modelos no nível de função, enquanto vulnerabilidades reais podem depender da interação entre arquivos, classes, módulos ou chamadas interprocedurais.

3.4 MODELOS PRÉ-TREINADOS PARA CÓDIGO

Uma linha mais recente utiliza modelos de linguagem pré-treinados especificamente em código-fonte. Esses modelos aprendem padrões a partir de grandes coleções de programas e podem ser adaptados para tarefas como classificação de código vulnerável, identificação de categorias da *Common Weakness Enumeration*

(CWE) e sugestão de correções.

O CodeBERT, proposto por Feng et al. (Feng *et al.*, 2020), é um modelo baseado em *transformers* que aprende representações conjuntas de código-fonte e linguagem natural. Após o ajuste fino (*fine-tuning*) com exemplos de funções vulneráveis e não vulneráveis, o modelo pode ser utilizado em tarefas de classificação de vulnerabilidades.

O GraphCodeBERT, proposto por Guo et al. (Guo *et al.*, 2021), amplia essa ideia ao incorporar informações de fluxo de dados. Dessa forma, o modelo busca preservar relações relevantes entre variáveis e operações, que podem não ser adequadamente representadas quando o código é tratado somente como uma sequência de tokens.

Essas soluções são promissoras porque conseguem lidar com dependências contextuais de longo alcance. Por exemplo, uma variável pode receber um valor em determinada parte do código e ser utilizada de forma insegura várias linhas depois. Um modelo baseado em *transformers* pode aprender relações desse tipo com maior eficiência do que métodos baseados apenas em frequência de palavras ou tokens.

No entanto, modelos baseados exclusivamente em sequências ainda podem perder informações estruturais relevantes. Por esse motivo, pesquisas recentes combinam *transformers* com representações em grafo, a fim de unir a compreensão contextual dos modelos de linguagem à representação estrutural oferecida pelos grafos de código (Guo *et al.*; Chakraborty *et al.*, 2021, 2021).

3.5 LARGE LANGUAGE MODELS E SOLUÇÕES HÍBRIDAS

Atualmente, os *Large Language Models* (LLMs) voltados à programação vêm sendo explorados para detectar vulnerabilidades, explicar os riscos encontrados e sugerir possíveis correções. Diferentemente de classificadores tradicionais, esses modelos podem analisar um trecho de código e gerar descrições em linguagem natural, indicando uma possível vulnerabilidade, sua categoria CWE e uma recomendação de mitigação.

Uma tendência recente é a integração de LLMs com *Code Property Graphs*. Nessa combinação, o grafo fornece informações estruturais, como fluxo de dados e fluxo de controle, enquanto o modelo de linguagem contribui com a interpretação contextual do código. Abordagens híbridas dessa natureza buscam melhorar a

capacidade de detectar vulnerabilidades que não são facilmente identificadas apenas pela análise de tokens ou por regras estáticas (Autor *et al.*; Autor *et al.*, 2026, 2025).

Apesar de seu potencial, os LLMs possuem limitações importantes. Eles podem gerar falsos positivos, deixar de identificar vulnerabilidades reais ou fornecer justificativas aparentemente plausíveis, mas incorretas. Também existem questões relacionadas ao custo computacional, à privacidade de código-fonte enviado a serviços externos e à dificuldade de reproduzir resultados quando modelos proprietários são utilizados.

Além disso, a capacidade de generalização ainda representa um desafio. Um modelo pode apresentar bons resultados em um conjunto de dados específico, mas obter desempenho inferior quando aplicado a projetos, linguagens ou distribuições de dados diferentes. Assim, a avaliação dessas soluções deve considerar não apenas métricas de classificação, mas também a taxa de falsos positivos, o tempo de processamento, a explicabilidade e a capacidade de generalização para código real (Li *et al.*; Chakraborty *et al.*, 2022, 2021).

3.6 ANÁLISE CRÍTICA DAS SOLUÇÕES EXISTENTES

A análise das soluções existentes mostra que não há uma abordagem única capaz de resolver completamente o problema de detecção de vulnerabilidades em código-fonte. As ferramentas de análise estática permanecem relevantes por sua maturidade, explicabilidade, rapidez e facilidade de integração aos processos de desenvolvimento. Entretanto, sua dependência de regras predefinidas limita a identificação de padrões novos ou complexos.

Os métodos tradicionais de *Machine Learning* representam uma alternativa às regras fixas, pois aprendem padrões a partir de dados. Contudo, sua eficácia depende da seleção de atributos e eles podem ter dificuldade para representar o contexto semântico e as relações de fluxo de dados presentes no programa.

As abordagens baseadas em *Deep Learning*, *transformers* e grafos de código apresentam maior potencial para modelar características sintáticas, semânticas e estruturais. Em particular, soluções híbridas que combinam modelos de linguagem e grafos de código são promissoras, pois integram a capacidade de compreensão contextual dos modelos de linguagem à representação explícita do fluxo de controle e

do fluxo de dados (Autor *et al.*; Autor *et al.*, 2026, 2025).

Entretanto, uma solução não deve ser considerada superior apenas por apresentar maior acurácia em um único conjunto de dados. A comparação entre abordagens deve considerar métricas como precisão, *recall*, *F1-score*, taxa de falsos positivos, custo computacional, tempo de inferência, explicabilidade e generalização para projetos não utilizados no treinamento. Também é necessário considerar a qualidade dos rótulos e o nível de análise adotado, pois muitos datasets avaliam apenas funções isoladas, enquanto vulnerabilidades reais podem depender de contexto interprocedural.

Dessa forma, as abordagens híbridas baseadas em modelos pré-treinados e grafos de código podem ser consideradas as mais promissoras para pesquisas atuais. No entanto, ferramentas estáticas continuam necessárias como apoio, principalmente em cenários nos quais a rastreabilidade do alerta, a privacidade do código e a execução rápida em pipelines de desenvolvimento são requisitos fundamentais.

3.7 SÍNTESE DO CAPÍTULO

Este capítulo apresentou as principais soluções utilizadas para a detecção de vulnerabilidades em código-fonte. Inicialmente, foram discutidas as ferramentas de análise estática, caracterizadas pela aplicação de regras e padrões predefinidos. Em seguida, foram apresentados os modelos tradicionais de *Machine Learning*, as abordagens de *Deep Learning*, os modelos de linguagem pré-treinados para código e os LLMs integrados a grafos de código.

Observou-se que as soluções mais recentes buscam combinar diferentes representações do código, principalmente informações textuais, sintáticas e semânticas. Embora modelos de linguagem e grafos apresentem potencial para detectar padrões complexos, desafios relacionados à qualidade dos dados, generalização, interpretabilidade e falsos positivos ainda permanecem.

Portanto, a proposta deste trabalho deve ser avaliada não apenas pelo desempenho em métricas tradicionais de classificação, mas também por sua capacidade de identificar vulnerabilidades em cenários realistas e por sua viabilidade de integração ao processo de desenvolvimento de software.

4 TRABALHO A SER DESENVOLVIDO

Objetivo : DESENVOLVER um software que analise...

4.1 DESCRIÇÃO FUNCIONAL E DIAGRAMA EM BLOCOS

4.2 DETALHAMENTO DOS MÓDULOS FUNCIONAIS

5 TECNOLOGIAS QUE SERÃO UTILIZADAS

5.1 TECNOLOGIAS ESCOLHIDAS E JUSTIFICATIVAS

5.2 TECNOLOGIAS ALTERNATIVAS AVALIADAS

6 PROCEDIMENTOS DE TESTE E VALIDAÇÃO DO PROJETO

6.1 PROCEDIMENTOS DE TESTE

6.2 CRITÉRIOS DE ACEITE E DESEMPENHO

6.3 ANÁLISE DOS RISCOS

Risco	Probab.	Impacto	Mitigação
Escassez ou desbalan- ceamento de dados ro- tulos de vulnerabili- dades	Média	Alto	Utilizar conjuntos de dados públicos já consolidados e técnicas de balanceamento de classes; complementar com dados sintéticos se necessário
Baixo desempenho do modelo de Machine Le- arning	Média	Alto	Validação cruzada, separação clara entre treino/teste e ajuste incremental dos modelos
Complexidade na inte- gração de Large Lan- guage Models ao <i>pipe- line</i> de análise	Média	Médio	Tratar o uso de LLMs como módulo complementar e opcional, priorizando primeiro um classificador de ML mais simples e funcional
Restrição de tempo do cronograma acadêmico	Alta	Alto	Priorizar entregas incrementais, com um MVP funcional antes de adicionar funcionalidades complementares
Limitações de poder computacional dos integrantes para treinamento de modelos	Baixa	Médio	Uso de infraestrutura em nuvem gratuita/educacional caso necessário
Falsos positivos elevados prejudicando a confiança na ferramenta	Média	Alto	Comparação contínua com ferramentas de análise estática tradicionais e ajuste dos limiares de classificação

7 CRONOGRAMA DO PROJETO

8 CONCLUSÃO

REFERÊNCIAS

- AUTOR, A. *et al.* Llmxcpg: Context-aware vulnerability detection through large language models and code property graphs. **arXiv preprint**, 2025.
- AUTOR, A. *et al.* Sv-gen: Synergizing llm-empowered variable semantics and graph transformers for vulnerability detection. **Future Internet**, v. 18, n. 5, 2026.
- CHAKRABORTY, S. *et al.* A survey on machine learning techniques for source code vulnerability detection. **arXiv preprint arXiv:2110.09610**, 2021.
- FENG, Z. *et al.* Codebert: A pre-trained model for programming and natural languages. In: **Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: Findings**. [S.l.: s.n.], 2020. p. 1536–1547.
- GUO, D. *et al.* Graphcodebert: Pre-training code representations with data flow. In: **International Conference on Learning Representations**. [S.l.: s.n.], 2021.
- LI, Y. *et al.* Machine learning for source code vulnerability detection. **IEEE Security & Privacy**, v. 20, n. 5, p. 39–48, 2022.
- MDPI. A survey on large language models in software security: Opportunities and threats. **Computers**, v. 15, n. 4, 2026. Disponível em: <https://www.mdpi.com/2073-431X/15/4/226>. Acesso em: 01 set. 2026.
- State of Cyber. **AI-Assisted Code Introduces 4.4x More Security Vulnerabilities Than Human-Written Code, New Research Finds**. 2026. The Tennessean. Disponível em: <https://www.tennessean.com/press-release/story/223693/ai-assisted-code-introduces-4-4x-more-security-vulnerabilities-than-human-written-code-new-research-finds/>. Acesso em: 01 set. 2026.
- TechScience. Static analysis techniques for secure software: A systematic review. **Journal of Cyber Security**, v. 7, n. 1, 2025. Disponível em: <https://www.techscience.com/JCS/v7n1/64045/html>. Acesso em: 01 set. 2026.
- WANG, Y. *et al.* A generative ai cybersecurity risks mitigation model for code generation: Using ann-ism hybrid approach. **Scientific Reports**, 2025. Disponível em: <https://www.nature.com/articles/s41598-025-34350-3>. Acesso em: 01 set. 2026.
- WU, Z. *et al.* **AI Code in the Wild: Measuring Security Risks and Mitigations**. 2025. ArXiv:2512.18567. Disponível em: <https://arxiv.org/abs/2512.18567>. Acesso em: 01 set. 2026.
- YANG, X. *et al.* **ZeroFalse: Improving Precision in Static Analysis with LLMs**. 2025. ArXiv:2510.02534. Disponível em: <https://arxiv.org/abs/2510.02534>. Acesso em: 01 set. 2026.
- ZHOU, Y. *et al.* Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks. In: **Advances in Neural Information Processing Systems**. [S.l.: s.n.], 2019. v. 32.